

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

**АЛГОРИТМЫ СОРТИРОВКИ, НЕ ИСПОЛЬЗУЮЩИЕ СРАВНЕНИЕ
ЭЛЕМЕНТОВ.**

ОТЧЁТ

студентки 3 курса 351 группы
направления 09.03.04 — Программная инженерия
факультета КНиИТ
Винник Маргариты Дмитриевны

Проверено:

доцент, к. ф.-м. н.

И. А. Батраева

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Реализация сортировки слиянием (Merge sort)	10
2 Реализация быстрой сортировки (Quicksort)	17
2.1 Теория	17
2.2 Классическая реализация быстрой сортировки с Lomuto partition :	22
2.3 Реализация рандомизированной быстрой сортировки Randomized QuickSort :	23
2.4 Реализация быстрой сортировки с разбиением Хоара Hoare partition scheme :	24
3 Реализация пирамидальной сортировки (сортировка кучи, Heapsort)	26
3.1 Теория	26
3.2 Реализация пирамидальной сортировки (Heapsort) на C++	28
4 Бенчмерки для всех трёх сортировок (собственный бенчмарк)	31
5 Оценки сложности сортировок	38
5.1 Сортировка слиянием (MergeSort)	38
5.2 Быстрая сортировка (рандомизированная, с трёхпутевым разбиением)	38
5.3 Пирамидальная сортировка (HeapSort)	39
6 Бенчмерки для всех трёх сортировок (бенчмарк, требующийся по заданию)	41
ЗАКЛЮЧЕНИЕ	49

ВВЕДЕНИЕ

Следует начать с того, что на данный момент у нас имеется богатый набор методов разработки алгоритмов. Одним из них является метод **декомпозиции**, или метод **"разделяй и властвуй"**, который нам понадобится для понимания и реализации сортировки слиянием и быстрой сортировки. Данный метод мы рассмотрим подробно именно на сортировке слиянием (Merge Sort), т.к. она в точности следует парадигме **"разделяй и властвуй"**.

Итак, начнём с общих принципов метода декомпозиции. Многие алгоритмы имеют рекурсивную структуру: для решения поставленной задачи они рекурсивно вызывают себя необходимое количество раз, решая вспомогательные подзадачи, тесно связанные с основной. Другими словами, сложная задача разбивается на несколько простых, которые подобны исходной задаче, но имеют меньший объём; далее эти вспомогательные задачи решаются рекурсивным методом, после чего полученные решения комбинируются для получения решения исходной задачи.

Парадигма, лежащая в основе метода декомпозиции "разделяй и властвуй" на каждом уровне рекурсии включает в себя три шага:

1. **Разделение** задачи на несколько подзадач, которые представляют собой меньшие экземпляры той же задачи.
2. **Властвование** над подзадачами путём их рекурсивного решения. Если размеры подзадач достаточно малы, такие подзадачи могут решаться непосредственно.
3. **Комбинирование** решений подзадач в решение исходной задачи.

Сразу рассмотрим алгоритм сортировки слиянием (Merge sort) как хороший и понятный пример применения парадигмы. Интуитивно он работает следующим образом:

1. **Разделение.** Делим n - элементную сортируемую последовательность на две последовательности по $n/2$ элементов.
2. **Властвование.** Рекурсивно сортируем эти две последовательности с использованием сортировки слиянием.
3. **Комбинирование.** Соединяем две отсортированные последовательности для получения окончательного отсортированного ответа.

Ключевая операция, которая производится в процессе сортировки слиянием - это объединение двух отсортированных последовательностей в ходе комби-

нирования (последний этап). Это делается с помощью вызова вспомогательной процедуры слияния **procedure MERGE(A, p, q, r)**, где A - входной массив, а p, q, r - индексы, нумерующие элементы массива, такие, что $p \leq q < r$. В этой процедуре предполагается, что элементы подмассивов $A[p \dots q]$ и $A[q + 1 \dots r]$ упорядочены.

Она *сливает* эти два подмассива в один отсортированный, элементы которого заменяют текущие элементы подмассива $A[p \dots r]$.

Для выполнения процедуры **MERGE(A, p, q, r)** требуется время $\Theta(n)$, где $n = r - p + 1$ - общее количество подлежащих слиянию элементов (позже мы это докажем).

Для наглядности процесса слияния возьмём как пример сортировку игральнх карт. Предположим, что на столе лежат две отсортированные стопки карт, обращённых лицевой стороной вниз, причём наверху находится карта наименьшего достоинства. Эти две стопки нужно объединить (слить) в одну выходную, в которой карты будут отсортированы и также будут обращены рубашкой вверх.

Основной шаг состоит в том, чтобы из двух младших карт выбрать самую младшую, извлечь её из соответствующей стопки (при этом в данной стопке верхней окажется новая карта) и поместить в выходную стопку. Этот шаг повторяется до тех пор, пока в одной из входных стопок не закончатся карты, после чего оставшиеся в другой стопке карты нужно поместить в выходную стопку.

Для подмассивов это будет выглядеть следующим образом: оба подмассива отсортированы в возрастающем порядке. Наименьший элемент находится в самом начале одного из них. Этот наименьший элемент перемещается в общий массив, при этом один из подмассивов укорачивается на один элемент. Следующий наименьший элемент также находится в одном из оставшихся двух подмассивов. Операция повторяется до тех пор, пока не опустеют оба отсортированных подмассива.

Реализация процедуры слияния на псевдокоде:

```

Merge(A, p, q, r)
1  n1 = q - p + 1
2  n2 = r - q
3  Пусть L[1...n1+1] и R[1...n2+1] - массивы
4  for i = 1 to n1
5      L[i] = A[p+i-1]
6  for j = 1 to n2
7      R[j] = A[q+j]
8  L[n1+1] = ∞
9  R[n2+1] = ∞
10 i = 1
11 j = 1
12 for k = p to r
13     if L[i] ≤ R[j]
14         A[k] = L[i]
15         i = i + 1
16     else A[k] = R[j]
17         j = j + 1

```

Дополнительное "усиление", позволяющее в ходе каждого основного шага не проверять явл. ли каждая из двух стопок пустой. В самый низ обеих объединяемых карт помещается так называемая **сигнальная карта** - ограничитель со значением ∞ . Это упрощает код и когда встречается сигнальная карта, меньше шансов не встретим до полного истощения обеих стопок.

Подробно рассмотрим работу процедуры **MERGE**:

- В строках №1, №2 вычисляются длины подмассивов $A[p \dots q]$, $A[q+1 \dots r]$ - n_1 , n_2 соответственно.
- В строке №3 создаются массивы L и R ("левый" и "правый") с длинами $n_1 + 1$, $n_2 + 1$ соответственно. Дополнительная ячейка массива будет содержать ограничитель.
- Строки №4-№7 с помощью циклов **for** копируют подмассивы $A[p \dots q]$, $A[q+1 \dots r]$ в массивы $L[1 \dots n_1]$ и $R[1 \dots n_2]$ соответственно.
- Строки №8, №9 помещают ограничители в последние ячейки L и R .
- Работу строк №10-17 проиллюстрируем. Они выполняют $r - p + 1$ базовый шаг, сохраняющий инвариант цикла.

	8	9	10	11	12	13	14	15	16	17					8	9	10	11	12	13	14	15	16	17				
$\mathcal{A}$	...	2	4	5	7	1	2	3	6	...					$\mathcal{A}$	...	1	4	5	7	1	2	3	6	...			
	k															k												
	1	2	3	4	5			1	2	3	4	5				1	2	3	4	5			1	2	3	4	5	
h	2	4	5	7	∞			R	1	2	3	6	∞		h	2	4	5	7	∞			R	1	2	3	6	∞
	i														i													
					(a)															(b)								
	8	9	10	11	12	13	14	15	16	17					8	9	10	11	12	13	14	15	16	17				
$\mathcal{A}$	...	1	2	5	7	1	2	3	6	...					$\mathcal{A}$	...	1	2	2	7	1	2	3	6	...			
	k															k												
	1	2	3	4	5			1	2	3	4	5				1	2	3	4	5			1	2	3	4	5	
h	2	4	5	7	∞			R	1	2	3	6	∞		h	2	4	5	7	∞			R	1	2	3	6	∞
	i														i													
					(b)															(c)								
	8	9	10	11	12	13	14	15	16	17					8	9	10	11	12	13	14	15	16	17				
$\mathcal{A}$	...	1	2	2	3	1	2	3	6	...					$\mathcal{A}$	...	1	2	2	3	4	2	3	6	...			
	k															k												
	1	2	3	4	5			1	2	3	4	5				1	2	3	4	5			1	2	3	4	5	
h	2	4	5	7	∞			R	1	2	3	6	∞		h	2	4	5	7	∞			R	1	2	3	6	∞
	i														i													
					(g)															(c)								
	8	9	10	11	12	13	14	15	16	17					8	9	10	11	12	13	14	15	16	17				
$\mathcal{A}$	...	1	2	2	3	4	5	3	6	...					$\mathcal{A}$	...	1	2	2	3	4	5	6	6	...			
	k															k												
	1	2	3	4	5			1	2	3	4	5				1	2	3	4	5			1	2	3	4	5	
h	2	4	5	7	∞			R	1	2	3	6	∞		h	2	4	5	7	∞			R	1	2	3	6	∞
	i														i													
					(ж)															(3)								
	8	9	10	11	12	13	14	15	16	17					8	9	10	11	12	13	14	15	16	17				
$\mathcal{A}$	...	1	2	2	3	4	5	6	7	...					$\mathcal{A}$	...	1	2	2	3	4	5	6	6	...			
	k															k												
	1	2	3	4	5			1	2	3	4	5				1	2	3	4	5			1	2	3	4	5	
h	2	4	5	7	∞			R	1	2	3	6	∞		h	2	4	5	7	∞			R	1	2	3	6	∞
	i														i													
					(и)															(3)								

Пояснение:

Операции в строках 10-17 процедуры MERGE ($A, 9, 12, 16$), когда подмассив $A[9 \dots 16]$ содержит последовательность $\langle 2, 4, 5, 7, 1, 2, 3, 6 \rangle$. После копирования и добавления ограничителей массив L содержит $\langle 2, 4, 5, 7, \infty \rangle$, а массив R содержит $\langle 1, 2, 3, 6, \infty \rangle$. Ячейки голубого цвета A содержат окончательные значения, а такие же голубые ячейки в L и R - значения, которые должны быть скопированы назад в A . Вместе голубые ячейки всегда включают значения, изначально находившиеся в $A[9 \dots 16]$, вместе с двумя ограничителями. Фиоле-

товые ячейки A содержат значения, в которые будет выполнено перезаписывающее копирование, а такие же фиолетовые ячейки L и R в содержат значения, уже скопированные обратно в A . (а)-(з) Массивы A , L и R и их индексы i , j , k перед каждой итерацией цикла в строках 12–17. (и) Массивы и индексы при завершении процедуры. В этот момент подмассив $A[9 \dots 16]$ отсортирован, а два ограничителя в L и R являются единственными элементами в этих массивах, которые не были скопированы в A .

— В начале каждой итерации цикла **for** в строках №12-17 подмассив

$A[p \dots k - 1]$ содержит $k - p$ наименьших элементов $L[1 \dots n_1 + 1]$ и $R[1 \dots n_2 + 1]$ в отсортированном порядке. Кроме того, $L[i]$ и $R[j]$ являются наименьшими элементами соответствующих массивов, которые ещё не были скопированы назад в A .

Покажем, что инвариант цикла соблюдается:

1. **Инициализация.** Перед первой итерацией цикла $k = p$, подмассив $A[p \dots k - 1]$ пуст. Он содержит $k - p = 0$ наименьших элементов массивов L и R . $i = j = 1 \Rightarrow$ элементы $L[i]$ и $R[j]$ - наименьшие элементы, не скопированные обратно в массив A .
2. **Сохранение.** Чтобы убедиться, что инвариант цикла сохраняется после каждой итерации, сначала предположим, что $L[i] \leq R[j]$. Тогда $L[i]$ - наименьший элемент, пока еще не скопированный в массив A . Поскольку в подмассиве $A[p \dots k - 1]$ содержится $k - p$ наименьших элементов, после копирования в строке 14 $L[i]$ в $A[k]$ в подмассиве $A[p \dots k - 1]$ будет содержаться $k - p + 1$ наименьших элементов. В результате увеличения параметра k цикла **for** и значения переменной i (строка 15), инвариант цикла восстанавливается перед следующей итерацией. Если же выполняется неравенство $L[i] > R[j]$, то в строках 16 и 17 выполняются соответствующие действия, в ходе которых также сохраняется инвариант цикла.
3. **Завершение.** Алгоритм завершается, когда $k = r + 1$. В соответствии с инвариантом цикла подмассив $A[p \dots k - 1]$ (т.е. подмассив $A[p \dots r]$) содержит $k - p = r - p + 1$ наименьших элементов массивов $L[1 \dots n_1 + 1]$ и $R[1 \dots n_2 + 1]$ в отсортированном порядке. Суммарное количество элементов в массивах L и R равно $n_1 + n_2 + 2 = r - p + 3$. Все они, кроме двух самых больших, скопированы обратно в массив

A , а два оставшихся элемента являются сигнальными.

Чтобы убедиться, что время работы процедуры MERGE равно $\Theta(n)$, где $n = r - p + 1$, заметим, что каждая из строк 1-3 и 8-11 выполняется за константное время; длительность цикла **for** в строках 4-7 равна $\Theta(n_1 + n_2) = \Theta(n)$, а в цикле **for** в строках 12-17 выполняются n итераций, на каждую из которых затрачивается константное время.

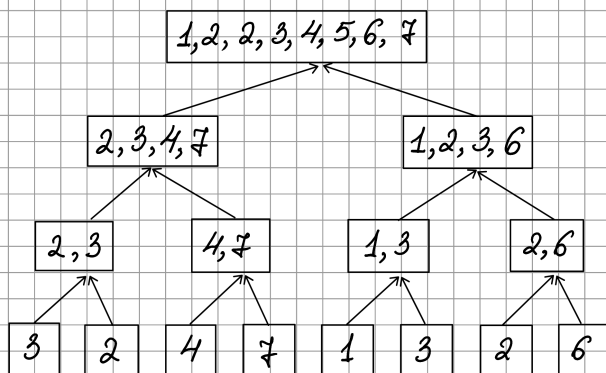
Теперь процедуру MERGE можно использовать в качестве подпрограммы в алгоритме сортировки слиянием. Процедура MERGE-SORT (A, p, r) выполняет сортировку элементов в подмассиве $A[p \dots r]$. Если справедливо неравенство $p \geq r$, то в этом подмассиве содержится не более одного элемента, и, т. о., он является уже отсортированным. В противном случае производится разбиение, в ходе которого вычисляется индекс q , разделяющий массив $A[p \dots r]$ на два подмассива: $A[p \dots q]$ с $\lfloor n/2 \rfloor$ элементами и $A[q + 1 \dots r]$ с $\lceil n/2 \rceil$ элементами.

Полный псевдокод:

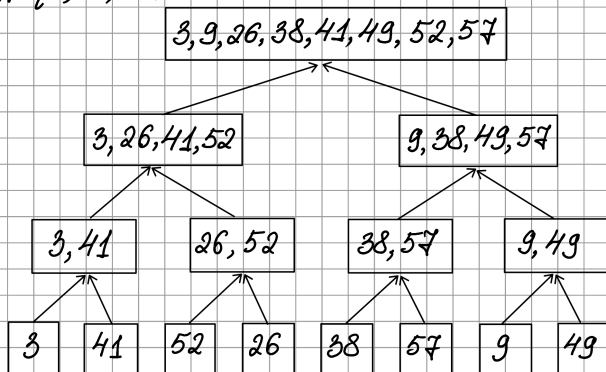
```

Merge( $A, p, q, r$ )
1-2   $n_1 = q - p + 1; n_2 = r - q$ 
3    создать  $h[1 \dots n_1 + 1]$  и  $R[1 \dots n_2 + 1]$ 
4    for  $i = 1$  to  $n_1$ 
5         $h[i] = A[p + i - 1]$ 
6    for  $j = 1$  to  $n_2$ 
7         $R[j] = A[q + j]$ 
8-9   $h[n_1 + 1] = \infty; R[n_2 + 1] = \infty$ 
10-11  $i = 1; j = 1$ 
13   for  $k = p$  to  $r$ 
14       if  $h[i] \leq R[j]$ 
15            $A[k] = h[i]$ 
16            $i = i + 1$ 
17       else  $A[k] = R[j]$ 
18            $j = j + 1$ 
Merge-Sort( $A, p, r$ )
1   if  $p < r$ 
2        $q = \lfloor (p + r) / 2 \rfloor$ 
3       Merge-Sort( $A, p, q$ )
4       Merge-Sort( $A, q + 1, r$ )
5       Merge( $A, p, q, r$ )
    
```

Процесс сортировки слиянием на 2х примерах:
 $A = \{5, 2, 4, 7, 1, 3, 2, 6\}$



$A = \{3, 41, 52, 26, 38, 57, 9, 49\}$



1 Реализация сортировки слиянием (Merge sort)

```
#include <iostream>
#include <vector>
#include <climits>
#include <cassert>
#include <algorithm>
#include <random>

using namespace std;

// MERGE(A, p, q, r) - по псевдокоду Кормена, переведено в 0-индексацию.
// A[p..r] включительно (здесь НЕ полуинтервал, а отрезок!)
// L[1..n1] и R[1..n2] в книге -> L[0..n1-1] и R[0..n2-1] здесь.
void merge(vector<int>& a, int p, int q, int r)
{
    int n1 = q - p + 1;
    int n2 = r - q;

    vector<int> L(n1 + 1);
    vector<int> R(n2 + 1);

    for (int i = 0; i < n1; i++)
        L[i] = a[p + i];
    for (int j = 0; j < n2; j++)
        R[j] = a[q + 1 + j];

    L[n1] = INT_MAX; // сигнальное значение вместо бесконечности
    R[n2] = INT_MAX;

    int i = 0, j = 0;
    for (int k = p; k <= r; k++)
    {
        if (L[i] <= R[j])
        {
            a[k] = L[i];
```

```

        i++;
    }
    else
    {
        a[k] = R[j];
        j++;
    }
}
}

// MERGE-SORT(A, p, r) - по псевдокоду, A[p..r] включительно
void mergeSortCLRS(vector<int>& a, int p, int r)
{
    if (p < r)
    {
        int q = (p + r) / 2;
        mergeSortCLRS(a, p, q);
        mergeSortCLRS(a, q + 1, r);
        merge(a, p, q, r);
    }
}

void printVec(const string& label, const vector<int>& v)
{
    cout << label << ": ";
    for (int x : v) cout << x << " ";
    cout << "\n";
}

void test_basic()
{
    // пример, который разбирает Корман
    vector<int> a = {5, 2, 4, 6, 1, 3};
    mergeSortCLRS(a, 0, (int)a.size() - 1);
    assert((a == vector<int>{1, 2, 3, 4, 5, 6}));
}

```

```

        cout << "[OK] test_basic\n";
    }

void test_single()
{
    vector<int> a = {9};
    mergeSortCLRS(a, 0, 0);
    assert((a == vector<int>{9}));
    cout << "[OK] test_single\n";
}

void test_already_sorted()
{
    vector<int> a = {1,2,3,4,5,6,7};
    mergeSortCLRS(a, 0, (int)a.size()-1);
    assert((a == vector<int>{1,2,3,4,5,6,7}));
    cout << "[OK] test_already_sorted\n";
}

void test_reverse_sorted()
{
    vector<int> a = {7,6,5,4,3,2,1};
    mergeSortCLRS(a, 0, (int)a.size()-1);
    assert((a == vector<int>{1,2,3,4,5,6,7}));
    cout << "[OK] test_reverse_sorted\n";
}

void test_duplicates()
{
    vector<int> a = {4,2,4,2,4,1,1};
    mergeSortCLRS(a, 0, (int)a.size()-1);
    assert((a == vector<int>{1,1,2,2,4,4,4}));
    cout << "[OK] test_duplicates\n";
}

```

```

void test_stability()
{
    // <= в строке 13 псевдокода - признак устойчивости
    // проверим на структуре
    struct P { int key, idx; };
    // кодируем след. образом: key*10000+idx
    // т.е. так, чтобы сравнение
    // по числу совпадало со сравнением по idx при равных key
    vector<int> keys = {3,1,3,2,1,3};
    vector<int> encoded;
    for (int i = 0; i < (int)keys.size(); i++)
        encoded.push_back(keys[i]*10000 + i);
    mergeSortCLRS(encoded, 0, (int)encoded.size()-1);
    vector<int> idxForKey3;
    for (int v : encoded)
    {
        if (v/10000==3)
        {
            idxForKey3.push_back(v%10000);
        }
    }
    // исходный порядок для ключа 3: индексы 0,2,5
    assert((idxForKey3 == vector<int>{0,2,5}));
    cout << "[OK] test_stability\n";
}

```

```

void test_random_stress(int iterations = 500)
{
    mt19937 rng(42);
    for (int it = 0; it < iterations; it++)
    {
        int n = rng() % 300;
        if (n == 0) continue;
        vector<int> a(n);
        for (auto& x : a) x = (int)(rng() % 2000) - 1000;
    }
}

```

```

        auto expected = a;
        sort(expected.begin(), expected.end());
        mergeSortCLRS(a, 0, n-1);
        assert(a == expected);
    }
    cout << "[OK] test_random_stress ("
    << iterations << " iterations)\n";
}

int main()
{
    test_basic();
    test_single();
    test_already_sorted();
    test_reverse_sorted();
    test_duplicates();
    test_stability();
    test_random_stress();
    cout << "\nВсе тесты пройдены успешно.\n";
}

```

Другая реализация (лекции Андросова):

```

#include <iostream>
#include <vector>
using namespace std;
vector<int> merge(vector<int> a, vector<int> b)
{
    int n = a.size();
    int m = b.size();
    vector<int> c(n + m);
    int i = 0;
    int j = 0;
    int k = 0;
    while (i != n && j != m)
    {

```

```

        if (a[i] <= b[j])
        {
            c[k] = a[i];
            k++;
            i++;
        }
        else
        {
            c[k] = b[j];
            k++;
            j++;
        }
    }
    while (i != n)
    {
        c[k] = a[i];
        k++;
        i++;
    }
    while (j != m)
    {
        c[k] = b[j];
        k++;
        j++;
    }
    return c;
}

vector<int> merge_sort(vector<int>& a)
{
    int n = a.size();
    if (n <= 1)
        return a;
    else
    {
        int m = n / 2;

```

```
vector<int> b(a.begin(), a.begin() + m);  
vector<int> c(a.begin() + m, a.end());  
return merge(merge_sort(b), merge_sort(c));  
}  
}
```

2 Реализация быстрой сортировки (Quicksort)

2.1 Теория

Данная сортировка подобно сортировке слиянием применяет парадигму "разделяй и властвуй".

Разберём и опишем процесс сортировки массива $A[l \dots r]$, состоящий из трёх этапов, как и все алгоритмы использующие принцип декомпозиции:

1. **Разделение.** Массив $A[l \dots r]$ разбивается на два (возможно, пустых) подмассива $A[l \dots \text{pivot} - 1]$ и $A[\text{pivot} + 1 \dots r]$, таких, что каждый элемент $A[l \dots \text{pivot} - 1]$ меньше или равен $A[\text{pivot}]$, который, в свою очередь, не превышает любой элемент подмассива $A[\text{pivot} + 1 \dots r]$. Индекс опорного элемента **pivot** вычисляется в ходе процедуры разбиения.
2. **Властвование.** Подмассивы $A[l \dots \text{pivot} - 1]$ и $A[\text{pivot} + 1 \dots r]$ сортируются с помощью рекурсивного вызова процедуры быстрой сортировки.
3. **Комбинирование.** Поскольку подмассивы сортируются на месте, для их объединения не требуются никакие действия: весь массив $A[l \dots r]$ оказывается отсортированным.

Но есть важный нюанс: быстрая сортировка очень чувствительна к выбору опорного элемента (**pivot**) и разделении массива на две (или три) части относительно этого выбранного элемента.

В общем виде алгоритм включает два ключевых решения:

1. **Выбор опорного элемента (pivot).** Классический подход Кормана предполагает выбор последнего элемента подмассива. Однако для защиты от худшего случая на отсортированных или почти отсортированных данных используется **рандомизированный выбор** (Randomized Quicksort), при котором **pivot** выбирается случайным образом из текущего диапазона.
2. **Процедура разбиения (Partition).** Существует два классических способа перестановки элементов вокруг **pivot**:
 - **Разбиение Ломута** (Lomuto partition) - простейший вариант, использующий один бегущий индекс;
 - **Разбиение Хоара** (Hoare partition) - оригинальный алгоритм, использующий два указателя, движущихся навстречу друг другу.

Быстрая сортировка реализуется следующей процедурой (псевдокод):

```

QuickSort( $A, l, r$ )
1  if  $l < r$ 
2      pivot( $A, l, r$ )
3      QuickSort( $A, l, \text{pivot} - 1$ )
4      QuickSort( $A, \text{pivot} + 1, r$ )

```

Как было упомянуто выше ключевой частью данной сортировки является выбор опорного элемента (**pivot**) и разбиение (**partition**) массива относительно этого элемента.

Псевдокод разбиения (**partition**):

```

Partition( $A, l, r$ )
1   $x = A[r]$ 
2   $i = l - 1$ 
3  for  $j = l$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          Обменять  $A[i]$  и  $A[j]$ 
7  Обменять  $A[i + 1]$  и  $A[r]$ 
8  вернуть  $i + 1$ 

```

Проиллюстрируем и подробно разберём работу **partition**:

		i	l, j						r
(a)		2	8	7	1	3	5	6	4
(б)		l, i	j						r
		2	8	7	1	3	5	6	4
(в)		l, i	j						r
		2	8	7	1	3	5	6	4
(г)		l, i	j						r
		2	8	7	1	3	5	6	4
(д)		l	i	j					r
		2	1	7	8	3	5	6	4
(е)		l	i	j					r
		2	1	3	8	7	5	6	4
(ж)		l	i	j					r
		2	1	3	8	7	5	6	4
(з)		l	i	j					r
		2	1	3	8	7	5	6	4
(и)		l	i	j					r
		2	1	3	4	7	5	6	8

Эта процедура всегда выбирает элемент $x = A[r]$ в качестве опорного (pivot). Разбиение подмассива $A[l \dots r]$ будет выполняться относительно этого элемента. В начале выполнения процедуры массив разделяется на четыре области (они могут быть пустыми). В начале каждой итерации цикла **for** в строках 3–6 каждая область удовлетворяет определённым свойствам, показанным на рисунке. Эти свойства можно сформулировать в виде инварианта цикла.

В начале каждой итерации цикла в строках 3–6 для любого индекса k массива справедливо следующее:

1. если $l \leq k \leq i$, то $A[k] \leq x$;
2. если $i + 1 \leq k \leq j - 1$, то $A[k] > x$;

3. если $k = r$, то $A[x] = x$.

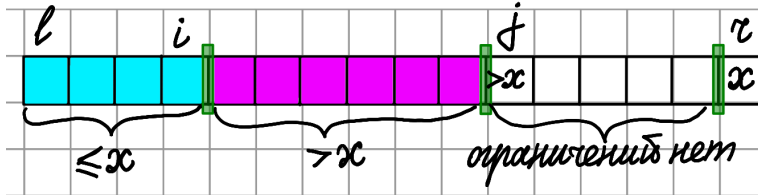
Индексы между j и $r - 1$ не подпадают ни под один из трех перечисленных случаев, и значения соответствующих им элементов не имеют определенной связи с опорным элементом x .

Докажем, что сформулированный выше инвариант цикла справедлив перед первой итерацией, что каждая итерация цикла сохраняет этот инвариант и что он позволяет продемонстрировать корректность алгоритма по завершении цикла:

1. **Инициализация.** Перед первой итерацией цикла $i = l - 1$ и $j = l$. Между элементами с индексами l и i нет никаких элементов, как нет их и между элементами с индексами $i + 1$ и $j - 1$, поэтому первые два условия инварианта цикла тривиально выполняются. Присваивание в строке 1 удовлетворяет третьему условию.
2. **Сохранение.** Как видно из рисунка, нужно рассмотреть два случая, выбор одного из которых определяется проверкой в строке 4. На рис. 7.3, (а) показано, что происходит, если $A[j] > x$; единственное действие, которое выполняется в этом случае в цикле, - это увеличение на единицу значения j . При увеличении значения j условие 2 выполняется для элемента $A[j - 1]$, а все остальные элементы остаются неизменными. На рисунке (б) показано, что происходит, если $A[j] \leq x$; в этом случае увеличивается значение i , элементы $A[i]$ и $A[j]$ меняются местами, после чего на единицу увеличивается значение j . В результате перестановки получаем $A[i] \leq x$, и условие 1 выполняется. Аналогично получаем $A[j - 1] > x$, поскольку элемент, который был переставлен в позицию элемента $A[j - 1]$, согласно инварианту цикла больше x .
3. **Завершение.** По завершении работы алгоритма $j = r$. Поэтому каждый элемент массива является членом одного из трех множеств, описанных в инварианте цикла. Таким образом, все элементы массива разбиты на три множества: величина которых не превышает x , превышающие x и одноэлементное множество, состоящее из элемента x . В последних двух строках процедуры Partition опорный элемент перемещается на свое место в середину массива с помощью его перестановки с крайним левым элементом, превышающим величину x . После этого процедура возвращает новый индекс опорного элемента. Выход процедуры Partition удовлетворяет тре-

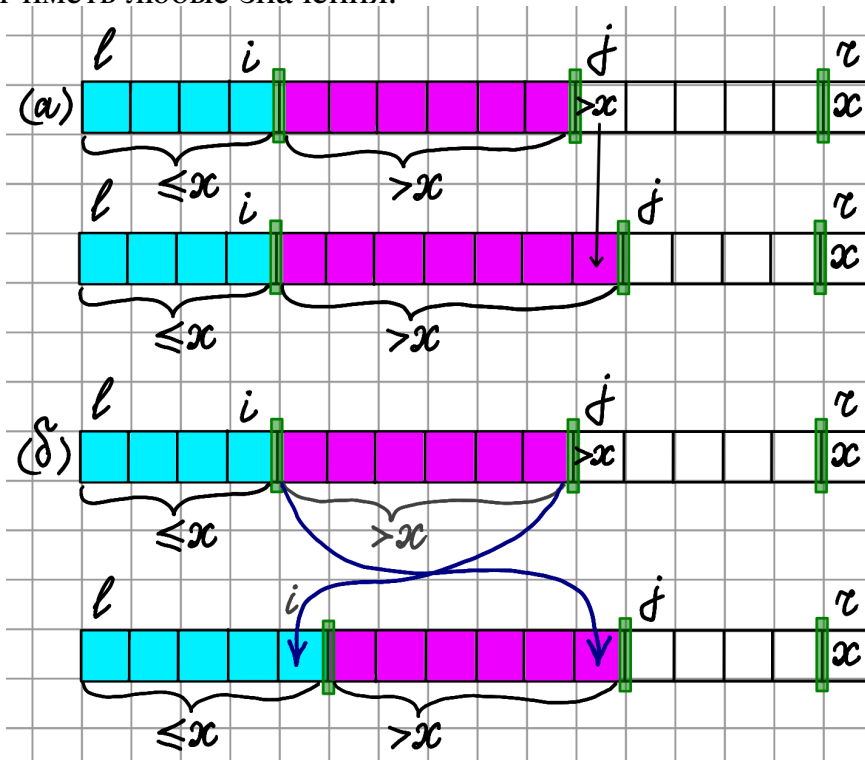
бованиям, наложенным шагом разделения. Фактически он удовлетворяет немного более строгому условию: после строки 2 процедуры QuickSort элемент $A[\text{pivot}]$ строго меньше любого элемента $A[\text{pivot} + 1 \dots r]$.

Проиллюстрируем:



Пояснение к картинке: Четыре области, поддерживаемые процедурой Partition в подмассиве $A[l \dots r]$.

Все элементы подмассива $A[l \dots i]$ меньше либо равны x , все элементы подмассива $A[i + 1 \dots j - 1]$ больше x , а $A[r] = x$. Подмассив $A[j \dots r - 1]$ может иметь любые значения.



Пояснение к картинке: Два варианта итерации в процедуре Partition. (а) Если $A[j] > x$, единственным действием является увеличение j , что сохраняет инвариант цикла. (б) Если $A[j] \leq x$, индекс i увеличивается, $A[i]$ и $A[j]$ меняются местами, после чего увеличивается j . Инвариант цикла сохраняется и в этом случае.

Полный псевдокод:

```

Partition(A, l, r)
1 x = A[r]
2 i = l - 1
3 for j = l to r - 1
4     if A[j] ≤ x
5         i = i + 1
6         Обменять A[i] и A[j]
7 Обменять A[i + 1] и A[r]
8 вернуть i + 1

QuickSort(A, l, r)
1 if l < r
2     pivot(A, l, r)
3     QuickSort(A, l, pivot - 1)
4     QuickSort(A, pivot + 1, r)

```

2.2 Классическая реализация быстрой сортировки с Lomuto partition:

```

#include <iostream>
#include <vector>
using namespace std;

int partition_Lomuto(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;

```

```

        swap(arr[i], arr[j]);
    }
}
swap(arr[i + 1], arr[high]);
return i + 1;
}

void quickSort_Lomuto(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition_Lomuto(arr, low, high);
        quickSort_Lomuto(arr, low, pi - 1);
        quickSort_Lomuto(arr, pi + 1, high);
    }
}

```

2.3 Реализация рандомизированной быстрой сортировки Randomized QuickSort:

```

#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <utility>

using namespace std;

mt19937 rnd(time(NULL));
int randInt(int l, int r)
{
    uniform_int_distribution<int>dist(l, r);
    return dist(rnd);
}

pair<int, int>partition_random(vector<int>& a, int l, int r)
{
    int pos = randInt(l, r - 1);

```

```

    int pivot = a[pos];
    int x = l, y = l;
    for (int i = l; i < r; i++)
    {
        if (a[i] <= pivot)
        {
            swap(a[i], a[y]);
            if (a[y] < pivot)
            {
                swap(a[y], a[x]);
                x++;
            }
            y++;
        }
    }
    return make_pair(x, y);
}

void quickSort_random(vector<int> &a, int l, int r)
{
    if ((r - l) < 2)
        return;
    pair<int, int> p = partition_random(a, l, r);
    quickSort_random(a, l, p.first);
    quickSort_random(a, p.second, r);
}

```

2.4 Реализация быстрой сортировки с разбиением Хоара Hoare partition scheme:

```

int hoarePartition(vector<int>& a, int low, int high)
{
    int pivot = a[low];
    int i = low - 1;
    int j = high + 1;
    while (true)

```

```

    {
        do { i++; } while (a[i] < pivot);
        do { j--; } while (a[j] > pivot);
        if (i >= j) return j;
        swap(a[i], a[j]);
    }
}

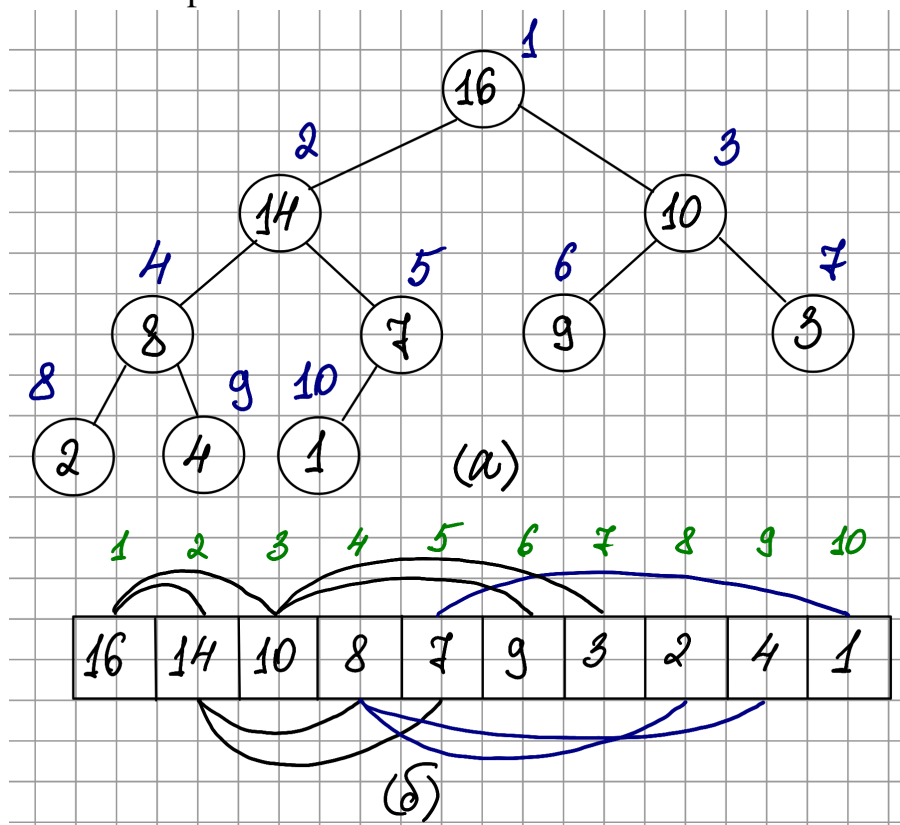
void quickSortHoare(vector<int>& a, int low, int high)
{
    if (low < high)
    {
        int p = hoarePartition(a, low, high);
        // ВАЖНО: [low, p], а не [low, p-1]!
        quickSortHoare(a, low, p);
        quickSortHoare(a, p + 1, high);
    }
}

```

3 Реализация пирамидальной сортировки (сортировка кучи, Heapsort)

3.1 Теория

В основе пирамидальной сортировки лежит структура данных пирамида или куча (**heap**). Начнём с неё. Структура данных (**бинарная**) **пирамида** представляет собой объект-массив, который можно рассматривать как почти полное бинарное дерево. Каждый узел этого соответствует элементу массива. Дерево полностью заполнено, за исключением, возможно, наинизшего, который заполняется слева направо.



Невозрастающая пирамида, представленная в виде (a) бинарного дерева и в виде (б) массива. Числа внутри кружков в каждом узле показывают значение, хранящееся в этом узле. Числа над узлами соответствуют индексам в массиве. Линии над и под элементами массива показывают отношения между родительскими и дочерними узлами; родительские узлы всегда находятся левее дочерних. Высота дерева равна трем; узел с индексом 4 (и значением 8) имеет высоту 1.

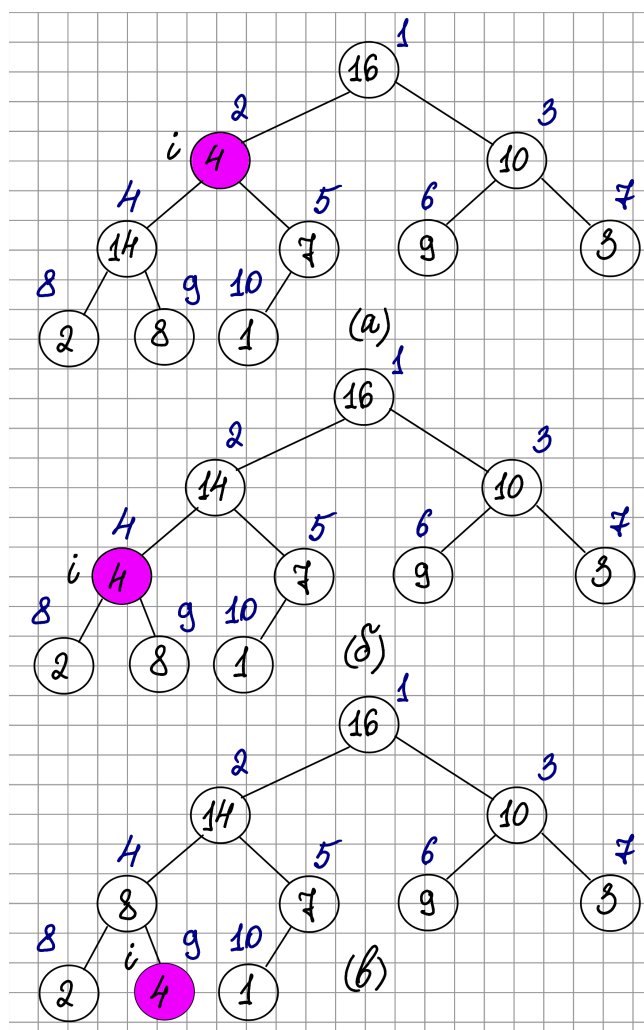
Массив A , представляющий пирамиду, является объектом с двумя атрибутами: $A.length$, который, как обычно, дает количество элементов в массиве, и $A.heap\text{-}size$, который указывает, сколько элементов пирамиды содержится в

массиве A . Т.е. хотя $A[1 \dots A.length]$ может содержать некоторые числа, только элементы подмассива $A[1 \dots A.heap-size]$, где $0 \leq A.heap-size \leq A.length$, являются корректными элементами пирамиды. Корнем дерева является $A[1]$, а для заданного индекса i узла можно легко вычислить индексы его родительского, левого и правого дочерних узлов.

Следует обратить внимание на поддержку свойств пирамиды.

Для поддержки свойства невозрастающей пирамиды вызывается процедура **Max-Heapify**. Ее входными данными являются массив A и индекс i в этом массиве. При вызове процедуры Max-Heapify предполагается, что бинарные деревья с корнями $LEFT(i)$ и $RIGHT(i)$ представляют собой невозрастающие пирамиды, но $A[i]$ может быть меньше, чем значения в дочерних узлах (таким образом, нарушая свойство невозрастающей пирамиды). Процедура Max-Heapify "сплавляет" ("просеивает") значение $A[i]$ вниз по невозрастающей пирамиде, так, что поддерево с корневым элементом с индексом i подчиняется свойству невозрастающей пирамиды.

Проиллюстрируем работу процедуры Max-Heapify:



Пояснение: Работа процедуры `Max-Heapify (A, 2)`, где `A.heap-size = 10`.
(а) Исходная конфигурация, в которой значение $A[2]$ в узле $i = 2$ нарушает свойство невозрастающей пирамиды, поскольку оно меньше, чем каждое из дочерних значений. Свойство невозрастающей пирамиды восстанавливается для узла 2 в части **(б)** путем обмена $A[2]$ с $A[4]$, который при этом нарушает свойство невозрастающей пирамиды для узла 4. В рекурсивном вызове `Max-Heapify (A, 4)` значение $i = 4$. После обмена $A[4]$ с $A[9]$, как показано в части **(в)**, ситуация в узле 4 исправляется, и рекурсивный вызов `Max-Heapify (A, 9)` не вносит никаких изменений в полученную структуру данных.

На каждом шаге определяется больший из элементов $A[i]$, $A[\text{LEFT}(i)]$ и $A[\text{RIGHT}(i)]$, и его индекс сохраняется в переменной **largest**. Если наибольшим оказывается $A[i]$, то поддерево с корнем i уже представляет собой корректную невозрастающую пирамиду и процедура завершает работу. В противном случае наибольшим оказывается один из двух дочерних элементов, и процедура выполняет обмен $A[i]$ с $A[\text{largest}]$, что приводит к тому, что для узла i и его дочерних узлов выполняется свойство невозрастающей пирамиды. Однако теперь исходное значение $A[i]$ оказывается в узле с индексом **largest**, так что поддерево с корнем **largest** может нарушать свойство невозрастающей пирамиды. Следовательно, необходимо рекурсивно вызвать процедуру `Max-Heapify` уже для этого дерева.

3.2 Реализация пирамидальной сортировки (Heapsort) на C++

```
#include <iostream>
#include <vector>
using namespace std;

// heapsort - сортировка, основанная на применении
// структуры данных пирамида
/*
* 1) Сначала строится невозрастающая пирамида,
* в корне кот. находится максимум
* 2) Каждый раз извлекается максимальный элемент
* и отправляется в конец. Операция происходит внутри
* самого массива, т.е. сортировка происходит in-place
* (не требует дополнительной памяти)
```

```

*/
// Свободные функции вместо методов struct heap в классической
// реализации для очереди с приоритетом куча (пирамида)
// работает напрямую на векторе, без обёртки в структуру
int parent(int x)
{
    return (x - 1) / 2;
}
int leftChild(int x)
{
    return 2 * x + 1;
}
int rightChild(int x)
{
    return 2 * x + 2;
}

void heapsort(vector<int>& a)
{
    int n = a.size();

    // стадия построения max-кучи: вставка по одному
    for (int i = 0; i < n; i++)
    {
        // нужно добавить i-й элемент в пирамиду
        // изначально i-й элемент на i-й позиции
        // необходимо его "просеять" вверх
        int j = i;
        while (j != 0 && a[parent(j)] < a[j])
        {
            swap(a[parent(j)], a[j]);
            j = parent(j);
        }
        // извлекли из пирамиды максимум
        // теперь она размера i, но в корне может быть

```

```

// элемент, который нарушает условие сохранения
// (поддержки) пирамиды
}

// стадия извлечения максимумов
for (int i = n - 1; i > 0; i--)
{
    swap(a[i], a[0]); // максимум (корень) уезжает в конец
    int x = 0;
    while (true)
    {
        int left = leftChild(x);
        int right = rightChild(x);
        int largest = x;
        if (left < i && a[left] > a[largest])
            largest = left;
        if (right < i && a[right] > a[largest])
            largest = right;
        if (largest == x)
            break;
        swap(a[x], a[largest]);
        x = largest;
    }
}
}

```

4 Бенчмерки для всех трёх сортировок (собственный бенчмарк)

По заданию требовалось прогнать алгоритмы и замерить время работы на массивах с четырьмя размерами (1000/10000/100000/1000000) и на трёх типах входных данных (просто чтобы интереснее было). Измерение времени работы алгоритмов производилось с помощью

```
std::chrono::high_resolution_clock
```

Для каждой комбинации размера массива и типа входных данных выполнялся единичный запуск каждого из трёх алгоритмов сортировки, время замерялось непосредственно вокруг вызова сортирующей функции (без учёта времени генерации входных данных). В качестве размеров массива использовались $n \in 1000, 10000, 100000, 1000000$. Для каждого размера рассматривалось три типа входных данных: случайно сгенерированный массив (генератор mt19937, равномерное распределение), уже отсортированный по возрастанию массив и массив, отсортированный по убыванию. Реализация быстрой сортировки использует рандомизированный выбор опорного элемента и трёхпутевое разбиение (Dutch National Flag), реализация сортировки слиянием и пирамидальной сортировки - согласно описанным выше алгоритмам.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
#include <chrono>
#include <climits>
#include <utility>
#include <ctime>
#include <iomanip>
using namespace std;
using Clock = chrono::high_resolution_clock;

// MergeSort (кормановская версия)
void merge(vector<int>& a, int p, int q, int r)
{
    int n1 = q - p + 1;
```

```

    int n2 = r - q;
    vector<int> L(n1 + 1), R(n2 + 1);
    for (int i = 0; i < n1; i++)
        L[i] = a[p + i];
    for (int j = 0; j < n2; j++)
        R[j] = a[q + 1 + j];
    L[n1] = INT_MAX;
    R[n2] = INT_MAX;
    int i = 0, j = 0;
    for (int k = p; k <= r; k++)
        a[k] = (L[i] <= R[j]) ? L[i++] : R[j++];
}

void mergeSortCLRS(vector<int>& a, int p, int r)
{
    if (p < r) {
        int q = (p + r) / 2;
        mergeSortCLRS(a, p, q);
        mergeSortCLRS(a, q + 1, r);
        merge(a, p, q, r);
    }
}

// Randomized QuickSort
mt19937 rnd_qs(time(NULL));
int randInt(int l, int r)
{
    uniform_int_distribution<int> dist(l, r);
    return dist(rnd_qs);
}

pair<int, int> partition_random(vector<int>& a, int l, int r)
{
    int pos = randInt(l, r - 1);
    int pivot = a[pos];
    int x = l, y = r;

```

```

    for (int i = l; i < r; i++)
    {
        if (a[i] <= pivot)
        {
            swap(a[i], a[y]);
            if (a[y] < pivot)
            {
                swap(a[y], a[x]);
                x++;
            }
            y++;
        }
    }
    return make_pair(x, y);
}

void quickSort_random(vector<int>& a, int l, int r)
{
    if ((r - l) < 2) return;
    pair<int, int> p = partition_random(a, l, r);
    quickSort_random(a, l, p.first);
    quickSort_random(a, p.second, r);
}

// HeapSort
int parent(int x)
{
    return (x - 1) / 2;
}

int leftChild(int x)
{
    return 2 * x + 1;
}

int rightChild(int x)
{
    return 2 * x + 2;
}

```

```

}
void heapsort(vector<int>& a)
{
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        int j = i;
        while (j != 0 && a[parent(j)] < a[j])
        {
            swap(a[parent(j)], a[j]);
            j = parent(j);
        }
    }
    for (int i = n - 1; i > 0; i--)
    {
        swap(a[i], a[0]);
        int x = 0;
        while (true)
        {
            int left = leftChild(x), right = rightChild(x);
            int largest = x;
            if (left < i && a[left] > a[largest])
                largest = left;
            if (right < i && a[right] > a[largest])
                largest = right;
            if (largest == x)
                break;
            swap(a[x], a[largest]);
            x = largest;
        }
    }
}

```

// Бенчмарк

```
double measure_merge(vector<int> a) {
```

```

    auto t1 = Clock::now();
    mergeSortCLRS(a, 0, (int)a.size() - 1);
    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

double measure_quick(vector<int> a) {
    auto t1 = Clock::now();
    quickSort_random(a, 0, (int)a.size());
    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

double measure_heap(vector<int> a) {
    auto t1 = Clock::now();
    heapsort(a);
    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

vector<int> genRandom(int n, mt19937& rng) {
    vector<int> a(n);
    for (auto& x : a) x = rng() % 1000000;
    return a;
}

vector<int> genSorted(int n) {
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = i;
    return a;
}

vector<int> genReverse(int n) {
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = n - i;
    return a;
}

int main()

```

```

{
    setlocale(LC_ALL, "ru");
    mt19937 rng(777);
    vector<int> sizes = { 1000, 10000, 100000, 1000000 };

    cout << left << setw(10) << "n" << setw(14) << "тип данных"
         << setw(14) << "Merge sort" << setw(18) << "Quicksort(rand)"
         << setw(12) << "Heapsort" << "\n";
    cout << string(68, '-') << "\n";

    for (int n : sizes)
    {
        vector<int> r = genRandom(n, rng);
        vector<int> s = genSorted(n);
        vector<int> rv = genReverse(n);

        cout << left << setw(10) << n << setw(14) << "случайный"
             << setw(14) << fixed << setprecision(2) << measure_merge(r)
             << setw(18) << measure_quick(r)
             << setw(12) << measure_heap(r) << "\n";

        cout << left << setw(10) << n << setw(14) << "отсортир."
             << setw(14) << measure_merge(s)
             << setw(18) << measure_quick(s)
             << setw(12) << measure_heap(s) << "\n";

        cout << left << setw(10) << n << setw(14) << "обратный"
             << setw(14) << measure_merge(rv)
             << setw(18) << measure_quick(rv)
             << setw(12) << measure_heap(rv) << "\n";
    }
    return 0;
}

```

n	тип данных	Merge sort	Quicksort(rand)	Heapsort
1000	случайный	1.35	0.54	0.67
1000	отсортир.	1.27	0.27	0.40
1000	обратный	1.24	0.25	0.25
10000	случайный	13.25	3.98	3.90
10000	отсортир.	12.68	2.79	5.69
10000	обратный	12.61	2.92	3.36
100000	случайный	138.77	38.94	48.96
100000	отсортир.	131.48	36.29	73.31
100000	обратный	130.99	35.46	42.25
1000000	случайный	1453.54	441.71	622.85
1000000	отсортир.	1383.77	396.48	888.40
1000000	обратный	1367.22	421.31	513.66

D:\source\бенчмарк\х64\Debug\бенчмарк.exe (процесс 9716) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:|

Таблица 4.1 – Сравнение времени работы сортировок (мс), сборка Debug

n	Тип данных	Merge sort	Quicksort (rand.)	Heapsort
1 000	случайный	1.35	0.54	0.67
1 000	отсортированный	1.27	0.27	0.40
1 000	обратный	1.24	0.25	0.25
10 000	случайный	13.25	3.98	3.90
10 000	отсортированный	12.68	2.79	5.69
10 000	обратный	12.61	2.92	3.36
100 000	случайный	138.77	38.94	48.96
100 000	отсортированный	131.48	36.29	73.31
100 000	обратный	130.99	35.46	42.25
1 000 000	случайный	1453.54	441.71	622.85
1 000 000	отсортированный	1383.77	396.48	888.40
1 000 000	обратный	1367.22	421.31	513.66

5 Оценки сложности сортировок

5.1 Сортировка слиянием (MergeSort)

Пусть $N = r - p + 1$ - размер сортируемого отрезка $a[p \dots r]$, элементарная операция - сравнение элементов при слиянии. Функция `merge` выполняет копирование во вспомогательные массивы и линейный проход слияния:

$$T_{merge}(n) = \Theta(n)$$

Рекурсивное разбиение отрезка пополам с последующим линейным слиянием даёт рекуррентное соотношение:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1)$$

По основной теореме о рекуррентных соотношениях (мастер-теорема) при $a = 2, b = 2, f(n) = \Theta(n)$:

$$n^{\log_b a} = n^{\log_2 2} = n = f(n)$$

выполняется случай 2 мастер-теоремы, откуда:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)$$

Разбиение отрезка на подзадачи выполняется строго по индексам (p, q, r) , без зависимости от значений элементов, поэтому оценка не зависит от входных данных:

$$T_a(n) = \Theta(n \log n) \quad (\text{худший} = \text{средний} = \text{лучший случай})$$

Дополнительная память: вспомогательный массив на слияние $S_d = \Theta(n)$, глубина рекурсии $S_r = O(\log n)$.

5.2 Быстрая сортировка (рандомизированная, с трёхпутевым разбиением)

Пусть $N = r - l$ - размер обрабатываемого отрезка. Функция `partition_random` выполняет один линейный проход с $O(1)$ работы на каждой итерации:

$$T_{part}(n) = \Theta(n)$$

Опорный элемент выбирается случайно и равновероятно из текущего отрезка, поэтому разбиение задаёт рекуррентное соотношение:

$$T(n) = \Theta(n) + T(k) + T(n - k - m)$$

где k — размер левой части, $m \geq 0$ — число элементов, равных опорному (они исключаются из дальнейшей рекурсии).

Худший случай. Опорный элемент каждый раз оказывается экстремумом текущего отрезка ($m = 0$):

$$T(n) = \Theta(n) + T(n - 1) + T(0) \implies T(n) = \Theta(n^2)$$

Лучший / сбалансированный случай. Опорный элемент делит отрезок примерно пополам:

$$T(n) = \Theta(n) + 2T\left(\frac{n}{2}\right) \implies T(n) = \Theta(n \log n)$$

Средний (ожидаемый) случай. В силу равновероятного выбора опорного элемента для *любого* входного массива математическое ожидание числа сравнений даёт:

$$E[T(n)] = \Theta(n \log n)$$

Таким образом:

$$T_a(n) = \begin{cases} \Theta(n^2), & \text{худший случай (маловероятен благодаря рандомизации)} \\ \Theta(n \log n), & \text{средний (ожидаемый) и лучший случай} \end{cases}$$

Дополнительная память: константный объём данных на вызов $S_d = O(1)$, глубина стека рекурсии $S_r = O(\log n)$ в среднем и $S_r = O(n)$ в худшем случае.

5.3 Пирамидальная сортировка (HeapSort)

Пусть $N = n = a.size()$. Алгоритм состоит из двух фаз.

Фаза 1 - построение кучи (просеивание вверх). При добавлении i -го элемента высота кучи составляет $\lfloor \log_2(i + 1) \rfloor$, просеивание в худшем случае

требует $O(\log i)$ сравнений:

$$T_1(n) = \sum_{i=1}^n O(\log i) = O(\log(n!)) = \Theta(n \log n) \quad (\text{худший случай})$$

$$T_1(n) = \Theta(n) \quad (\text{лучший случай — куча уже почти построена})$$

Фаза 2 - извлечение максимумов (просеивание вниз). На i -й итерации куча имеет размер i , просеивание занимает $O(\log i)$ сравнений:

$$T_2(n) = \sum_{i=1}^{n-1} O(\log i) = O(\log((n-1)!)) = \Theta(n \log n)$$

причём эта оценка достигается независимо от входных данных.

Итоговая трудоёмкость:

$$T_a(n) = T_1(n) + T_2(n) = \Theta(n \log n) \quad (\text{худший} = \text{средний} = \text{лучший случай})$$

Сортировка выполняется на месте (in-place), дополнительная память:

$$S_d = O(1)$$

6 Бенчмерки для всех трёх сортировок (бенчмарк, требующийся по заданию)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <random>
#include <chrono>
#include <climits>
#include <utility>
#include <ctime>

using namespace std;
using Clock = chrono::high_resolution_clock;

void merge(vector<int>& a, int p, int q, int r)
{
    int n1 = q - p + 1;
    int n2 = r - q;
    vector<int> L(n1 + 1), R(n2 + 1);
    for (int i = 0; i < n1; i++)
        L[i] = a[p + i];
    for (int j = 0; j < n2; j++)
        R[j] = a[q + 1 + j];
    L[n1] = INT_MAX; R[n2] = INT_MAX;
    int i = 0, j = 0;
    for (int k = p; k <= r; k++)
    {
        if (L[i] <= R[j])
        {
            a[k] = L[i];
            i++;
        }
        else
```

```

        {
            a[k] = R[j];
            j++;
        }
    }
}

void mergeSortCLRS(vector<int>& a, int p, int r)
{
    if (p < r) {
        int q = (p + r) / 2;
        mergeSortCLRS(a, p, q);
        mergeSortCLRS(a, q + 1, r);
        merge(a, p, q, r);
    }
}

// RANDOMIZED QUICKSORT (трёхпутевая)
mt19937 rnd_qs(time(NULL));
int randInt(int l, int r)
{
    uniform_int_distribution<int> dist(l, r);
    return dist(rnd_qs);
}

pair<int, int> partition_random(vector<int>& a, int l, int r)
{
    int pos = randInt(l, r - 1);
    int pivot = a[pos];
    int x = l, y = l;
    for (int i = l; i < r; i++)
    {
        if (a[i] <= pivot)
        {
            swap(a[i], a[y]);
            if (a[y] < pivot)

```

```

        {
            swap(a[y], a[x]); x++;
        }
        y++;
    }
}

return make_pair(x, y);
}

void quickSort_random(vector<int>& a, int l, int r)
{
    if ((r - l) < 2) return;
    pair<int, int> p = partition_random(a, l, r);
    quickSort_random(a, l, p.first);
    quickSort_random(a, p.second, r);
}

// HEAPSORT
int parent(int x)
{
    return (x - 1) / 2;
}

int leftChild(int x)
{
    return 2 * x + 1;
}

int rightChild(int x)
{
    return 2 * x + 2;
}

void heapsort(vector<int>& a)
{
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        int j = i;

```

```

    while (j != 0 && a[parent(j)] < a[j])
    {
        swap(a[parent(j)], a[j]);
        j = parent(j);
    }
}
for (int i = n - 1; i > 0; i--)
{
    swap(a[i], a[0]);
    int x = 0;
    while (true)
    {
        int left = leftChild(x), right = rightChild(x),
        largest = x;
        if (left < i && a[left] > a[largest])
            largest = left;
        if (right < i && a[right] > a[largest])
            largest = right;
        if (largest == x) break;
        swap(a[x], a[largest]);
        x = largest;
    }
}
}

// Бенчмарк: 5 запусков, среднее
double measure_merge(vector<int> a) {
    auto t1 = Clock::now();
    mergeSortCLRS(a, 0, (int)a.size() - 1);
    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

double measure_quick(vector<int> a) {
    auto t1 = Clock::now();
    quickSort_random(a, 0, (int)a.size());
}

```

```

    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

double measure_heap(vector<int> a) {
    auto t1 = Clock::now();
    heapsort(a);
    auto t2 = Clock::now();
    return chrono::duration<double, milli>(t2 - t1).count();
}

vector<int> genRandom(int n, mt19937& rng) {
    vector<int> a(n);
    for (auto& x : a) x = rng() % 1000000;
    return a;
}

int main()
{
    setlocale(LC_ALL, "ru");
    mt19937 rng(2026);
    vector<int> sizes = { 50000, 100000, 500000, 1000000 };
    const int RUNS = 5;

    ofstream csv("results.csv");
    csv << "n,algorithm,avg_time_ms\n";

    cout << "n\t\tMerge sort\tQuicksort(rand)\tHeapsort\n";
    cout << "-----\n";

    for (int n : sizes)
    {
        double sumMerge = 0, sumQuick = 0, sumHeap = 0;
        for (int run = 0; run < RUNS; run++)
        {
            vector<int> base = genRandom(n, rng);

```

```

        sumMerge += measure_merge(base);
        sumQuick += measure_quick(base);
        sumHeap += measure_heap(base);
    }

    double avgMerge = sumMerge / RUNS;
    double avgQuick = sumQuick / RUNS;
    double avgHeap = sumHeap / RUNS;

    cout << n << "\t\t" << avgMerge << "\t\t" << avgQuick
    << "\t\t" << avgHeap << "\n";

    csv << n << ",Merge sort," << avgMerge << "\n";
    csv << n << ",Quicksort," << avgQuick << "\n";
    csv << n << ",Heapsort," << avgHeap << "\n";
}
csv.close();
cout << "\nРезультаты сохранены в results.csv\n";
return 0;
}

```

n	Merge sort	Quicksort(rand)	Heapsort
50000	80.2943	21.6829	26.6247
100000	181.58	46.2769	59.9666
500000	836.475	257.728	313.49
1000000	1620.77	516.62	687.763
Результаты сохранены в results.csv			

Таблица 6.1 – Среднее время выполнения сортировок по 5 запускам (мс)

n	Merge sort	Quicksort (rand.)	Heapsort
50 000	80.29	21.68	26.62
100 000	181.58	46.28	59.97
500 000	836.48	257.73	313.49
1 000 000	1620.77	516.62	687.76

Код для построения графика (ура, питончик!):

```
import matplotlib.pyplot as plt
```

```

import pandas as pd

df = pd.read_csv("D:/source/bench_mark2/results.csv")

plt.figure(figsize=(8, 5.5))

markers = {"Merge sort": "o", "Quicksort": "s", "Heapsort": "^"}
colors = {"Merge sort": "#d62728", "Quicksort": "#1f77b4",
"Heapsort": "#2ca02c"}

for algo in df["algorithm"].unique():
    sub = df[df["algorithm"] == algo].sort_values("n")
    plt.plot(sub["n"], sub["avg_time_ms"], marker=markers[algo],
            color=colors[algo], linewidth=2,
            markersize=7, label=algo)

plt.xlabel("Размер массива (n)", fontsize=12)
plt.ylabel("Среднее время выполнения (мс)", fontsize=12)
plt.title("Зависимость времени выполнения сортировок \
от размера массива",
fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.ticklabel_format(style='plain', axis='x')
plt.tight_layout()
plt.savefig("benchmark_plot.png", dpi=150)
plt.savefig("benchmark_plot.pdf")
print("Сохранено: benchmark_plot.png и benchmark_plot.pdf")

```

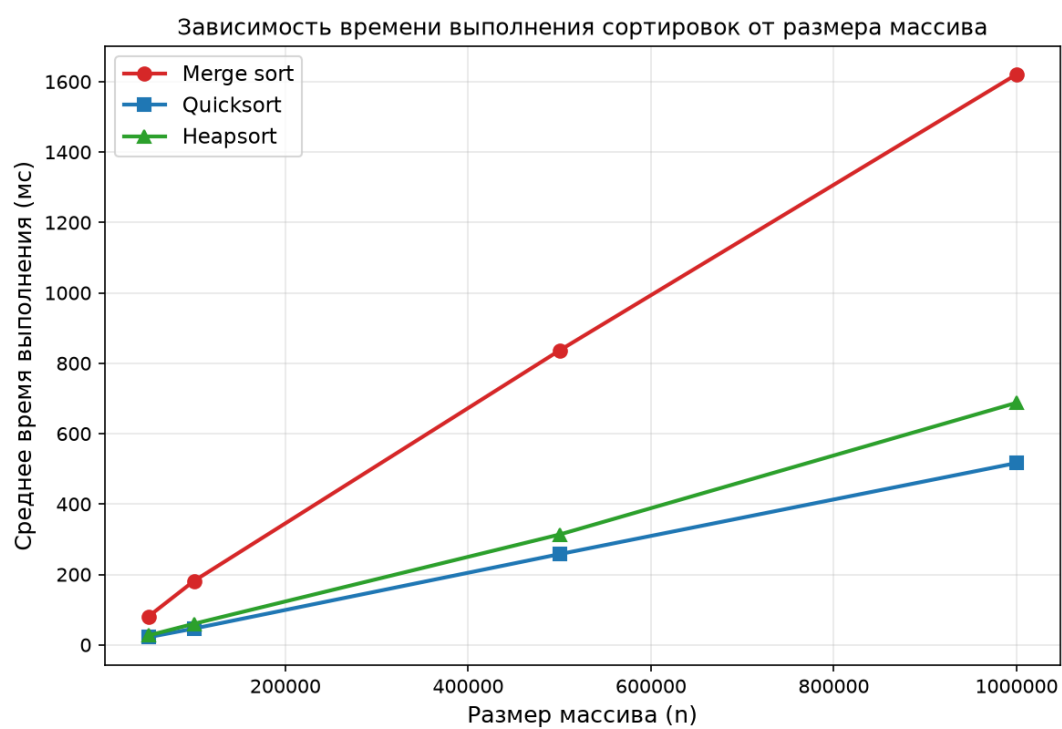


Рисунок 6.1 – Зависимость времени выполнения сортировок от размера массива

ЗАКЛЮЧЕНИЕ

Выводы по моему собственному бенчмарку:

Экспериментальные результаты подтверждают теоретические оценки сложности: все три алгоритма демонстрируют рост времени работы, согласующийся с асимптотикой $\Theta(n \log n)$ на случайных данных. Измерения проводились в конфигурации **Debug** (без оптимизаций компилятора), поэтому абсолютные значения времени существенно выше, чем можно было бы получить в **Release**-сборке, однако относительное соотношение между алгоритмами остаётся показательным.

Быстрая сортировка с рандомизированным выбором опорного элемента оказалась самой быстрой во всех замерах, опережая сортировку слиянием в 2.5 – 3.5 раза на большинстве размеров (например, при $n = 1000000$: 441.71 мс против 1453.54 мс на случайных данных). Это объясняется тем, что она работает **in-place**, не требует дополнительной памяти на каждом уровне рекурсии в отличие от сортировки слиянием, и обладает меньшей константой в асимптотике за счёт простоты внутреннего цикла разбиения. Рандомизация выбора опорного элемента устраняет квадратичную деградацию на отсортированных входных данных, характерную для классической реализации с фиксированным выбором **pivot**: время работы на всех трёх типах входных данных остаётся сопоставимым (например, при $n = 1000000$: 441.71/396.48/421.31 мс).

Сортировка слиянием стабильно показала наибольшее время работы среди трёх алгоритмов на всех размерах и типах данных, что связано с накладными расходами на выделение дополнительной памяти для временных массивов на каждом уровне рекурсии - суммарный объём временных аллокаций за весь запуск составляет $\Theta(n \log n)$, а не $\Theta(n)$, как может показаться на первый взгляд.

Пирамидальная сортировка показала неожиданную асимметрию в зависимости от типа входных данных: на уже отсортированном по возрастанию массиве она оказалась заметно медленнее, чем на случайных данных (888.40 мс против 622.85 мс при $n = 1000000$), тогда как на массиве, отсортированном по убыванию, - заметно быстрее (513.66 мс). Это объясняется устройством используемой стадии построения кучи методом последовательной вставки с просеиванием вверх (`sift_up`): на возрастающем входном массиве каждый новый добавляемый элемент оказывается больше всех предыдущих и гарантированно "всплывает" до корня кучи, выполняя $O(\log i)$ операций при каждой

вставке - это близко к худшему случаю для данной стадии. На убывающем массиве, напротив, каждый новый элемент оказывается меньше уже вставленных, и просеивание вверх завершается мгновенно за $O(1)$. Случайный порядок даёт промежуточный результат, поскольку лишь часть вставок требует длинного просеивания. Таким образом, в отличие от сортировки слиянием и быстрой сортировки, эффективность данной реализации пирамидальной сортировки существенно зависит не только от размера, но и от направления упорядоченности входных данных.

Таким образом, эмпирические измерения в целом согласуются с теоретическими оценками асимптотической сложности алгоритмов, а также демонстрируют, что константный множитель, скрытый в асимптотической записи, и особенности конкретной реализации (в частности, метод построения кучи) могут существенно влиять на практическую производительность алгоритмов с одинаковой формальной сложностью.

Выводы по бенчмарку из задания:

Результаты тестирования (табл. и рис.) подтверждают теоретическую оценку сложности $\Theta(n \log n)$ для всех трёх алгоритмов: график зависимости времени выполнения от размера массива демонстрирует характерный для этой асимптотики рост без резких скачков и без признаков квадратичной зависимости. Измерения проводились в конфигурации сборки **Debug** (без оптимизаций компилятора), что даёт заметно большие абсолютные значения времени по сравнению с оптимизированной сборкой, однако не влияет на относительное соотношение между алгоритмами.

Быстрая сортировка с рандомизированным выбором опорного элемента оказалась самой быстрой на всех рассмотренных размерах массива, стабильно опережая пирамидальную сортировку в 1.2 – 1.3 раза (например, при $n = 1000000$: 516.62 мс против 687.76 мс) и сортировку слиянием - в 3.1 – 3.9 раза. Такое устойчивое преимущество объясняется тем, что быстрая сортировка работает **in-place** и обладает низкой константой в асимптотике за счёт простого и предсказуемого для процессора внутреннего цикла разбиения, а рандомизация выбора опорного элемента исключает квадратичную деградацию производительности на неудачно упорядоченных входных данных.

Пирамидальная сортировка заняла промежуточное положение между быстрой сортировкой и сортировкой слиянием на всех размерах массива, стабильно

уступая быстрой сортировке примерно на 20-30%. Оба алгоритма работают **in-place** и не требуют дополнительной памяти, однако пирамидальная сортировка обращается к памяти по индексам дерева кучи, что даёт менее последовательный (менее кэш-дружественный) паттерн доступа по сравнению с линейным проходом при разбиении в быстрой сортировке - это, по-видимому, и обуславливает стабильно более высокое время выполнения.

Сортировка слиянием на всех размерах массива показала наибольшее время выполнения, отставая от быстрой сортировки в 3—4 раза, а от пирамидальной - в 2.4 — 3.0 раза. Основная причина - дополнительные затраты памяти на создание временных массивов L и R на каждом уровне рекурсии: суммарный объём временных аллокаций за весь запуск составляет $\Theta(n \log n)$, тогда как быстрая и пирамидальная сортировки обходятся без дополнительной памяти, сортируя массив на месте.

Таким образом, эмпирические результаты полностью согласуются с теоретическими оценками асимптотической сложности и наглядно демонстрируют, что при формально одинаковой сложности $\Theta(n \log n)$ практическая производительность алгоритмов существенно определяется особенностями работы с памятью конкретной реализации: необходимостью дополнительных аллокаций, характером обращений к памяти и, как следствие, эффективностью использования кэша процессора. Наилучший практический результат среди рассмотренных алгоритмов показала рандомизированная быстрая сортировка.